Python Portfolio: Konzeption

IU Internationale Hochschule

1 Ziele

Ziel des Dashboards ist es, den eigenen Studienfortschritt zielorientiert darzustellen. Im Fokus stehen dabei einige Kennzahlen, die eine schnelle Einschätzung des Status ermöglichen.

1.1 Primäre Ziele

1. Das Studium soll innerhalb einer Regelstudienzeit von 3 Jahren erfolgreich abgeschlossen werden.
2. Das Studium soll mit einem Notendurchschnitt von 2,0 oder besser abgeschlossen werden.

2 Inhalte des Dashboards

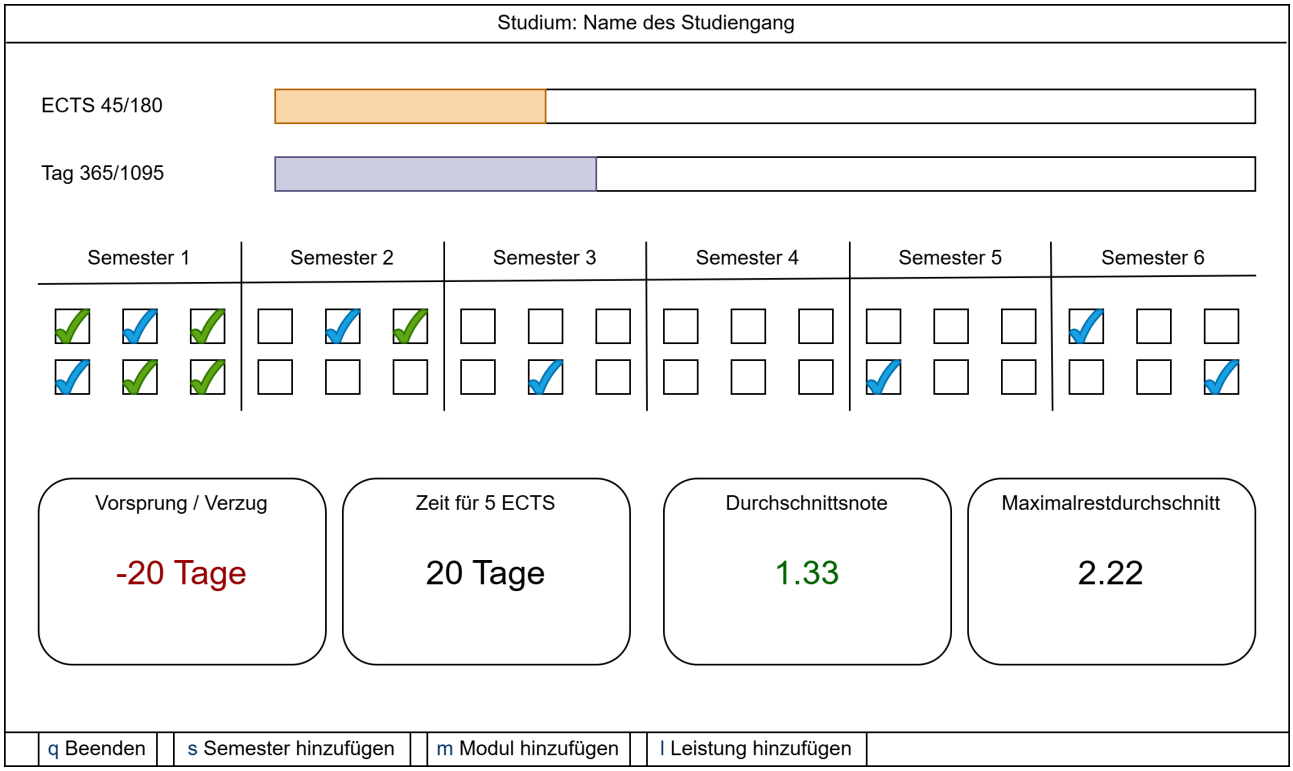

Abbildung 1: UI-Mockup

Das Dashboard soll als TUI (Terminal User Interface) umgesetzt werden. Dies erlaubt die Anwendung auch in Umgebungen ohne grafischer Oberfläche auszuführen.

Anschließend werden die Inhalte des Dashboards von oben nach unten beschrieben.

2.1 Kopfzeile

Die Kopfzeile beinhaltet nur den Namen des Studiengangs und trägt hauptsächlich zum Designkonzept der Anwendung bei.

2.2 Fortschrittsbalken

Unterhalb der Kopfzeile befinden sich zwei Fortschrittsbalken. Der erste beschreibt, wie viele ECTS bereits erreicht wurden. Der zweite beschreibt den zeitlichen Fortschritt der Studienzeit, basierend auf dem deklarierten Zielende.

Die beiden Fortschrittsbalken erlauben eine schnelle optische Gegenüberstellung der beiden Kennzahlen und helfen das Zeitziel einzuhalten.

2.3 Semesterübersicht

In der Semesterübersicht werden in einer Tabelle Checkboxen für alle Module dargestellt. Grüne Haken bedeuten hierbei, dass das Modul erfolgreich abgeschlossen wurde. Blaue Haken beschreiben Module, die aufgrund Vorerfahrung, angerechnet wurden.

2.4 Infokarten

Die vier Infokarten beinhalten wichtige Schlüsselwerte, die sowohl zur Erreichung des Zeitziels, als auch des Notenziels dienen.

Die erste Karte links beinhaltet den Vorsprung oder Verzug in Tagen, die durch das Vorzeichen unterschieden werden können.

Die Karte rechts davon beschreibt, wie viele Tage zum aktuellen Zeitpunkt pro 5 ECTS zur Verfügung stehen. Dabei wurde bewusst 5 ECTS ausgewählt, da die meisten Module zu einem Erwerb von 5 ECTS führen. Allerdings gibt es auch Module mit 10 ECTS, daher wurde die Notation 5 ECTS statt „Modul“ gewählt.

Rechts daneben befindet sich die dritte Karte, die den aktuellen Notendurchschnitt darstellt. Dies ist wichtig um einzuschätzen, ob die Zielnote erreicht wird und ob Maßnahmen zur Verbesserung der Noten notwendig sind.

Die letzte Karte beinhaltet die Kennzahl des Maximalrestdurchschnitts. Der Wert beschreibt den maximalen Notendurchschnitt der mit den restlichen Prüfungen erreicht werden muss, um die Zielnote zu erreichen.

2.5 Fußzeile

Die Fußzeile beinhaltet vier Buttons, die zur Steuerung der Anwendung notwendig sind. Hier können Semester, Module und Prüfungsleistungen hinzugefügt werden. Zusätzlich wird die entsprechende Tastenbelegung angezeigt. Zum Beispiel kann durch drücken der Taste **q** die Applikation beendet werden.

3 UML-Klassendiagramm

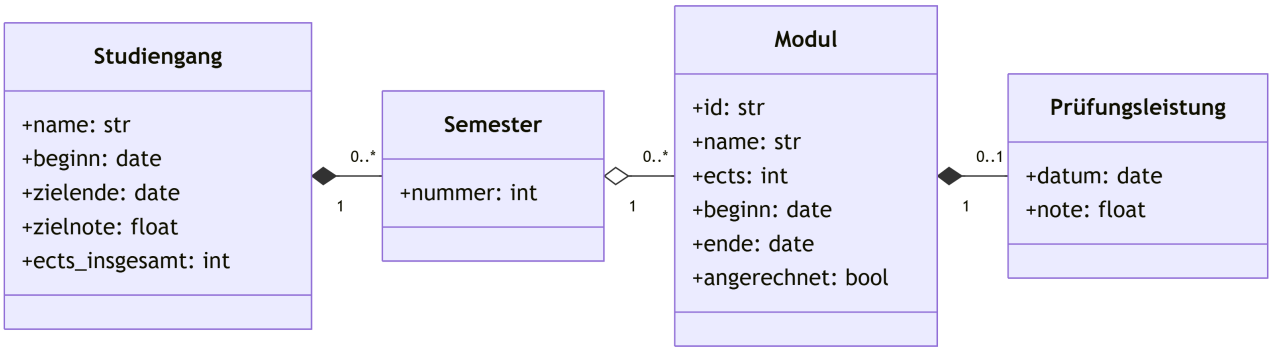

Abbildung 2: UML-Diagramm

Das Klassendiagramm besteht aus den Klassen Studiengang, Semester, Modul und Prüfungsleistung. Ein Studiengang hat viele Semester. Diese Beziehung wurde für diese Aufgabe als Komposition modelliert. Diese Entscheidung wurde getroffen, da die Semester-Klasse für sich wenig Aussagekraft besitzt und lediglich die Semesternummer enthält.

Ein Semester besitzt wiederum viele Module, wobei diese Beziehung hier als Aggregation festgelegt wurde. Hintergrund dafür ist die Annahme, dass ein Modul für sich genügend Aussagekraft besitzt, um abseits eines Semesters zu existieren.

Jedes Modul kann optional genau eine Prüfungsleistung besitzen. Alternativ könnten hier auch mehrere Prüfungsleistungen zugelassen werden. Dagegen wurde sich jedoch entschieden, um die Logik einfach zu halten. Zudem hat ein Fehlversuch keine Auswirkung auf den Notenschnitt, da die Prüfung sowieso wiederholt werden muss. Bei der Beziehung handelt es sich um eine Komposition mit einem optionalen Objekt. Dies wurde so gewählt da eine Prüfungsleistung ohne Modul ohnehin nicht zugeordnet werden kann und somit unsinnig wäre.

Zur Diskussion stand die Limitation der Semester- und Modulanzahl auf 6 Objekte, jedoch wurde sich dagegen entschieden, da nicht sicher davon ausgegangen werden kann, dass diese Anzahl für alle Studiengänge gilt. Die Anwendung soll auch ohne Anpassungen mit mehreren Semestern und Modulen betrieben werden können.

Auf eine abstrakte Basisklasse wurde in der Konzeptionsphase bewusst verzichtet, da die Klassen klar abgegrenzt sind und aktuell keine gemeinsamen Eigenschaften benötigen.

4 Machbarkeitsüberprüfung

4.1 Programmiersprache und Entwicklungsumgebung

Zur Umsetzung der Dashboard Anwendung soll Python 3.14 verwendet werden, da diese Version aktuell die neuste stabile Version des Interpreters ist und gute Unterstützung für objektorientierte

Konzepte bietet. Als IDE soll für diese Aufgabe Visual Studio Code mit einem Plugin für Python verwendet werden.

4.2 Bibliotheken

Zur Darstellung der TUI (Terminal User Interface) wird **textual** verwendet, da diese Bibliothek einen objektorientierten Ansatz verfolgt, der gut mit der geplanten Anwendung einher geht.

Abseits davon werden einige Standardbibliotheken, wie **json** für den Umgang mit JSON-Dateien und **dataclasses**, verwendet.

4.3 Datenspeicherung

Die Datenspeicherung soll in einer JSON-Datei erfolgen, da sich JSON-Dateien leicht versenden und sichern lassen. Zudem ist die Implementation einfach. Im Folgenden wird erprobt, wie eine JSON-Datei aus einer verschachtelten Klasse erstellt und anschließend wieder eingelesen werden kann:

```python
from dataclasses import dataclass, asdict
import json

@dataclass
class Semester:
    nummer: int

@dataclass
class Studiengang:
    name: str
    semester: list[Semester]

def speichere_studiengang(studiengang: Studiengang, dateiname: str) -> None:
    with open(dateiname, "w", encoding="utf-8") as f:
        json.dump(asdict(studiengang), f, ensure_ascii=False, indent=2)

def lade_studiengang(dateiname: str) -> Studiengang:
    with open(dateiname, "r", encoding="utf-8") as f:
        daten = json.load(f)
    return Studiengang(**daten)

if __name__ == "__main__":
    original = Studiengang("Angewandte künstliche Intelligenz", [ Semester(1) ])
    speichere_studiengang(original, "studium.json")

    geladen = lade_studiengang("studium.json")
    print(geladen)
```

Listing 1: Daten speichern & laden in JSON-Datei

4.4 Quellcodeverwaltung

Zur Quellcodeverwaltung soll Git mit GitHub als Git-Server verwendet werden. Durch Git Commits werden die einzelnen Entwicklungsschritte ersichtlich und mit weiteren Features wie Branches, können neue Funktionen sauber zum Quellcode hinzugefügt werden.

4.5 Benutzerschnittstelle

Die Anwendung soll als TUI (Terminal User Interface) implementiert werden, dies ermöglicht eine breite Verfügbarkeit der Anwendung. Die Anwendung kann auch auf Maschinen ohne grafischer Benutzeroberfläche bedient werden, wie zum Beispiel Linux Servern.